다듬 5단계 설명

같은 프로그램을 **초등학생** → **중학생** → **고등학생** → **대학생** → **전문가** 순서로 점점 깊게 설명합니다. 앞 단계가 이해되면 다음 단계로 가세요. 아무 단계에서 멈춰도 그 단계의 설명은 완결됩니다.

예문은 계속 이 문장입니다.

> **학교을 가는길에 친구를 만낫는데 그렇게 하면 안되요.**

1단계. 초등학생

다듬은 선생님 21명이 한 글을 같이 채점하는 교실입니다.

1. 글을 내면 선생님마다 **자기 과목만** 봅니다. 받침 선생님은 `을/를`만, 띄어쓰기 선생님은 빈칸만, 사전 선생님은 "자주 틀리는 말"만 봅니다.
2. 틀린 곳에 **스티커**를 붙이고, 고칠 말을 적습니다.
3. 같은 자리에 스티커가 겹치면 **반장**이 하나만 남깁니다.
4. 스티커는 두 가지입니다. **확실한 건 실선**(누르면 바로 고쳐짐), **애매한 건 점선**(설명만).

제일 중요한 규칙: 모르면 가만히 있기. 맞는 글을 틀리게 고치면 사람들이 한 번에 도구를 꺼 버리기 때문입니다.

```
학교을 → 학교를      (실선)
안되요 → 안 돼요     (실선)
가는길에              (점선: 띄어야 할 수도 있어요)
만낫는데              (점선: 이상한 말 같아요, 뭘로 고칠지는 모름)
```

2단계. 중학생

프로그램은 두 부분입니다.

```
[화면]  웹 · 크롬 확장 · 맥 앱   → 글을 보내고 밑줄을 그림
   │  요청: {"text": "..."}
   ▼
[서버]  파이썬 엔진              → 지적 목록을 돌려줌
```

식당으로 치면 **주방은 하나(서버), 테이블은 셋(화면)**입니다. 전부 내 컴퓨터 안에서 돌아서 글이 밖으로 나가지 않습니다.

엔진 안은 3단계입니다.

단계	하는 일
① 분석	문장을 형태소(뜻의 최소 조각)로 쪼갬
② 후보	검사기 21개가 각자 "여기 틀림"을 냄
③ 병합	겹치는 후보를 우선순위로 하나만 남김

검사기는 세 종류입니다: **규칙**(사람이 조건을 적음), **모델**(예시로 배움), **조언**(수상하다고만 말함).

예문에서 실제로 나온 후보는 10개였고 병합 후 8개가 남았습니다. 예를 들어 `되요`는 사전 규칙과 모델이 둘 다 지적했는데, **사전 규칙이 우선순위 5, 모델이 4**라서 사전 규칙이 남았습니다.

3단계. 고등학생

받침은 산수

한글 11,172자는 `(초성×21+중성)×28+종성` 순서로 번호가 매겨져 있어서 `(글자번호−44032) % 28` **이 받침 번호**(0이면 없음)입니다. `교`는 0이라 `를`이어야 하는데 `을`이 쓰였으니 오류입니다. `로/으로`만 ㄹ 받침(8번)을 받침 없음처럼 취급합니다(`서울로`).

Kiwi를 믿지 못하는 자리

`학교을`의 `을`을 Kiwi는 조사가 아니라 **명사**로 읽습니다. 그래서 "조사 태그를 찾는" 대신 **명사 바로 다음 글자**를 직접 봅니다.

병합 알고리즘

```
1. 후보를 (시작 위치, 긴 것 먼저) 순으로 정렬
2. 겹치지 않으면 남긴다
3. 겹치면: ① 고칠 말이 있는 쪽 → ② 출처 우선순위 높은 쪽 → ③ 먼저 온 쪽
```

①이 있는 이유: 이상탐지(고칠 말 없음)가 `자률신경` 전체를 잡아 그 안의 `률→율` 교정이 사라진 사고가 있었기 때문입니다.

고친 뒤 위치 이동

한 글자가 늘면(`안`→`안`) 뒤쪽 지적의 위치를 `delta`만큼 밉니다. 실제로 `매읽`의 위치가 30에서 31로 옮겨졌습니다.

4단계. 대학생

학습 모델: 글자 조각 + 로지스틱 회귀

1. **슬롯**: 헷갈리는 자리를 정규식으로 찾음(`되/돼`).
2. **특징**: 슬롯을 `▒`로 가리고 앞뒤 12글자를 잘라 옴 → `그렇게 하면 안 ▒요.`
3. **조각**: 2~4글자 조각을 전부 뽑아 TF-IDF로 가중(`안`, `▒요`, …).
4. **판정**: `z = Σ(조각 값×가중치)+절편`, `P = 1/(1+e^−z)`.

실제 손계산(배포된 모델 파일을 직접 읽음): `안 ▒요` → P(되)=0.095, P(돼)=0.905. `▒요` 조각이 −0.36으로 가장 크게 `돼` 쪽으로 밉니다. `predict_proba`와 일치했습니다.

학습 데이터는 공짜

맞게 쓴 위키 문장에서 슬롯을 가리면 **원문 글자가 정답**입니다. 틀린 문장을 모을 필요가 없습니다. 시험용은 **문장 단위**로 분리합니다(같은 문장의 슬롯이 양쪽에 들어가면 성적이 부풀려짐).

문턱

정문 오탐 ≤0.5%를 지키는 **가장 낮은 확률 문턱**을 측정이 고릅니다. 못 맞추면 모델을 싣지 않습니다.

규칙이 먼저

`되` + `고/게/면…`는 무조건 `되`, 어절 끝 `되`는 `돼`, 관형사형 `든`은 `던` — 이런 형태론 게이트가 모델보다 먼저 확정합니다.

평가

지표	값
정밀도	99.2%
정문 오탐	0.00%
실사용 자동교정	74 / 72 / **67%** (dev / holdout / 처음 본 문장)

5단계. 전문가

설계 결정

- **단일 흐름**: API와 평가가 모두 `engine.analyze()`만 부른다. 과거 두 군데에 복사돼 문체 재현율이 평가에서 0으로 찍힌 적이 있다.
- **출처 표 한 곳**: `sources.py` 58개. 표에 없는 출처가 나오면 테스트가 실패한다.
- **문장 단위 판정**: 문서 안과 밖에서 결과가 달라지면 안 된다(P5 불변식).
- **학습/서빙 동일 코드**: 슬롯·특징 함수를 `app/`에 두고 `ml/`이 가져다 쓴다(train/serve skew 차단).
- **`.ddm`**: 어휘 blob + offset + idf + coef를 이진 탐색으로 읽는다. 117MB → 5.7MB, 변환 시 sklearn과 확률 대조(최대 5.55e-16).
- **해시 사전**: 충돌은 "모르는 말을 안다고 착각"하는 안전한 방향(미탐)으로만 틀린다.
- **언어모델은 순위만**: 후보를 만들지 못하게 해서 새 오탐이 측정 밖에서 생기지 않게 한다.

경계 모델

Kiwi가 제안한 자리마다 P(띄움)을 판정한다. 문턱 0.97(진짜 경계의 9.3% 재현, 문장 오탐 0.03%). **8글자 이상 + 조사·어미 뒤 내용이 2회 이상**인 dense 덩어리는 모델이 위치를 구별하지 못해 Kiwi를 믿고 희귀 조각 사이만 버린다. 데이터를 5번 늘려도 재현율이 제자리 → 한계는 데이터가 아니라 모델 종류(bag-of-ngrams).

평가 체계

`layers → per-rule fpr → coverage → 단위 테스트 1,020 → 모델·사전 대조 → QA(P1–P5, 회귀 22, 퍼즈) → 하한`. 지표보다 **"이 숫자를 믿어도 되는가"**를 먼저 묻는 순서가 설계다. 표본 <20이면 수치를 내지 않는다.

알려진 약점 (직접 관찰)

항목	내용
같은 오타, 문맥 의존	`나는 매읽 운동한다` 는 Kiwi가 `매/NNG+읽/VV` 로 분석해 오타 검사가 침묵
문서-코드 불일치	README 436 vs 474 규칙, 1.5초 vs 0.7초 대기
환경 의존	태거(torch)·이상탐지 통계(.gitignore)는 환경에 따라 꺼짐
평가의 한계	실사용 셋 주석자 1명, 약 500문장